

E-Com Database Detailed Summary

Project scanned: C:/E-Com/backend/eCom. This PDF focuses only on the database model: entities, columns, relationships, repository queries, data lifecycle, ER diagram, and database risks.

1. Database Overview

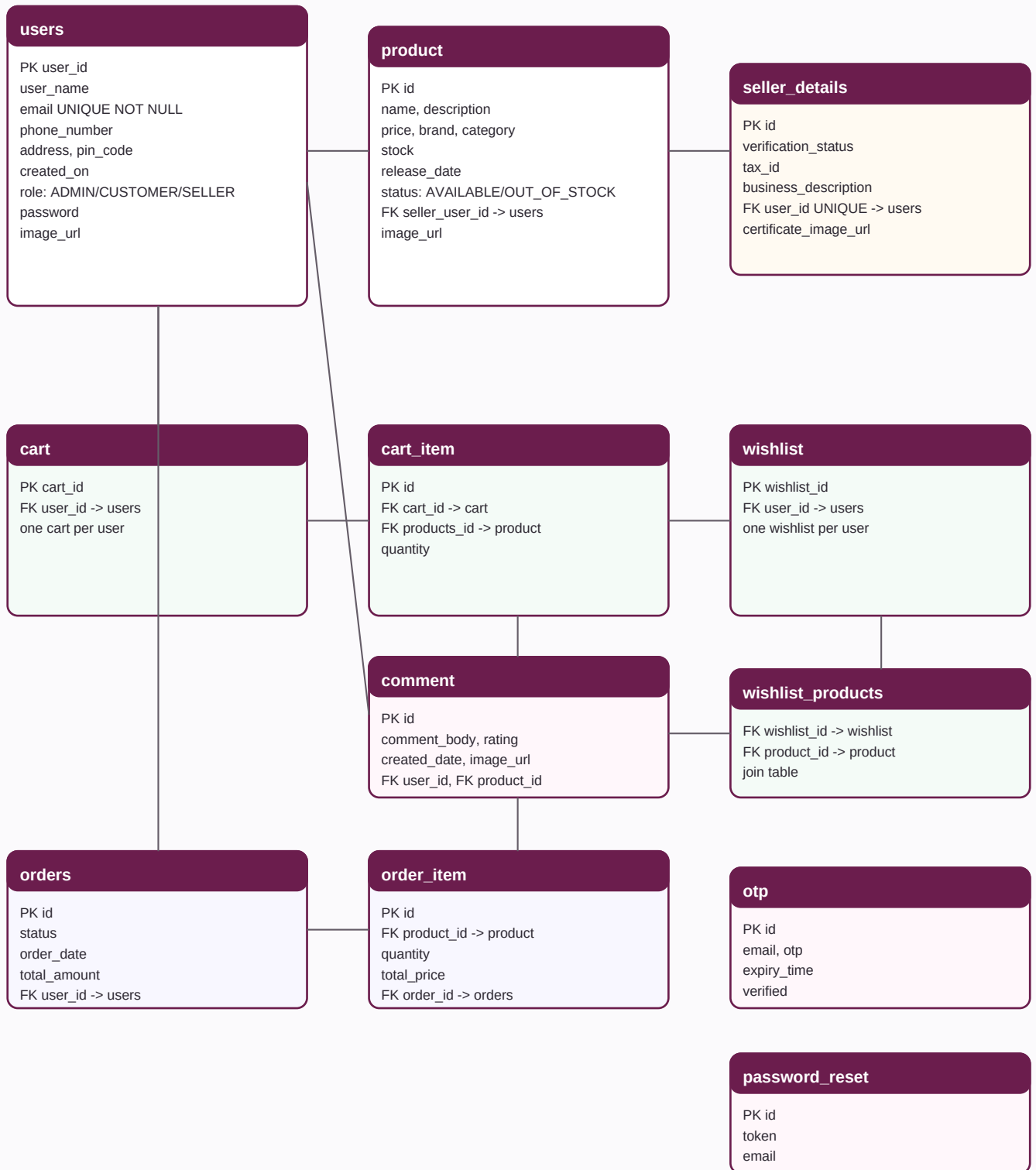
- The backend uses PostgreSQL through Spring Data JPA/Hibernate.
- The active database is configured in application.properties with a Neon PostgreSQL JDBC URL.
- Hibernate ddl-auto is set to update, so entity changes can automatically modify the schema during development.
- The model uses JPA entities with Lombok for boilerplate code generation.
- Most image values are stored as URLs in text columns rather than binary blobs.
- The database supports users, products, seller verification, carts, cart items, orders, order items, wishlists, comments, OTP codes, and password reset tokens.

2. ER Diagram

The diagram below shows the main tables/entities and their relationships as inferred from the JPA annotations and repository methods.

Mercato Database ER Diagram

Cardinality labels show the relationship direction used by the JPA model.



3. Entity and Table Details

users

Column / Field	Type / Constraint	Purpose
user_id	Long, primary key, identity	Main account identifier.
user_name	String	Display/name field for users.
email	String, unique, not null	Login identifier and account contact email.
phone_number	String	Customer/seller phone number.
address	String	Saved user address.
pin_code	int	Postal/pin code.
created_on	LocalDate	Account creation date, also used by seller cleanup logic.
role	Enum string	ADMIN, CUSTOMER, or SELLER.
password	String	BCrypt encoded password.
image_url	String	Profile image URL.

product

Column / Field	Type / Constraint	Purpose
id	Long, primary key, identity	Product identifier.
name	String	Product name.
description	String	Product details.
price	double	Current unit price.
brand	String	Brand name, also used in search.
category	String	Category filter value.
stock	int	Available inventory quantity.
release_date	Date	Product release/listing date.
status	Enum string	AVAILABLE or OUT_OF_STOCK, usually derived from stock.
seller_user_id	FK to users	Seller who owns this product. JPA creates FK from Product.seller.
image_url	String	Product image URL.

seller_details

Column / Field	Type / Constraint	Purpose
id	Long, primary key, identity	Seller details identifier.
verification_status	Enum string	PENDING or APPROVED.
tax_id	String	Seller tax identifier.
business_description	String	Seller business summary.
user_id	One-to-one FK, unique	Links details to exactly one seller user.
certificate_image_url	String	Uploaded certificate/tax document URL.

cart

Column / Field	Type / Constraint	Purpose
cart_id	Long, primary key, identity	Cart identifier.
user_id	One-to-one FK to users	Owner of the cart.

cart_item

Column / Field	Type / Constraint	Purpose
id	Long, primary key, identity	Cart line identifier.
cart_id	Many-to-one FK to cart	Cart containing this item.
products_id	Many-to-one FK to product	Product in the cart. Column name is products_id in the annotation.
quantity	int	Quantity in cart.

orders

Column / Field	Type / Constraint	Purpose
id	Long, primary key, identity	Order identifier.
status	Enum string	PENDING, PLACED, CONFIRMED, SHIPPED, DELIVERED, CANCELLED, RETURNED
order_date	LocalDate	Date order was created.
total_amount	double	Total order price.
user_id	Many-to-one FK to users	Customer who placed the order.

order_item

Column / Field	Type / Constraint	Purpose
id	Long, primary key, identity	Order line identifier.
product_id	Many-to-one FK to product	Purchased product.
quantity	int	Purchased quantity.
total_price	double	Line total at order time.
order_id	Many-to-one FK to orders	Parent order.

wishlist

Column / Field	Type / Constraint	Purpose
wishlist_id	Long, primary key, identity	Wishlist identifier.
user_id	One-to-one FK to users, not null	Owner of wishlist.

wishlist_products

Column / Field	Type / Constraint	Purpose
wishlist_id	FK to wishlist	Join table reference to wishlist.
product_id	FK to product	Join table reference to product.

comment

Column / Field	Type / Constraint	Purpose
id	long, primary key, identity	Comment/review identifier.
comment_body	String	Review text.
user_id	Many-to-one FK to users	Reviewer.
product_id	Many-to-one FK to product	Reviewed product.
created_date	LocalDate	Review creation date.
rating	double	Numeric rating used for product average.
image_url	String	Optional review image URL.

otp

Column / Field	Type / Constraint	Purpose
id	Long, generated	OTP record identifier.
email	String	Email being verified.
otp	String	Six-digit OTP code.
expiry_time	LocalDateTime	Expiry timestamp, currently 5 minutes after generation.
verified	boolean	Flag exists, but verification flow deletes the OTP instead of setting it true.

password_reset

Column / Field	Type / Constraint	Purpose
id	Long, primary key, identity	Reset record identifier.
token	String UUID	Password reset token.
email	String	Account email associated with token.

4. Relationship Details

Relationship	Meaning	Implementation
users -> product	One seller user can own many products.	Product.seller uses @ManyToOne.
users -> seller_details	One user can have one seller verification profile.	SellerDetails.user has unique user_id.
users -> cart	One user can have one cart.	User.cart is mappedBy user; Cart.user is @OneToOne.
cart -> cart_item	One cart has many cart items.	Cascade all and orphanRemoval true.
product -> cart_item	One product can appear in many cart items.	CartItem.product is @ManyToOne.
users -> orders	One user can place many orders.	User.orders is @OneToMany; Order.user is @ManyToOne.
orders -> order_item	One order has many order items.	Cascade all and orphanRemoval true.
product -> order_item	One product can appear in many order items.	OrderItem.product is @ManyToOne.
users -> comment	One user can write many comments.	User.comments is @OneToMany; Comment.user is @ManyToOne.
product -> comment	One product can have many comments.	Product.comments cascades all and removes orphan comments.
users -> wishlist	One user can have one wishlist.	Wishlist.user is @OneToOne and not null.
wishlist -> product	Many-to-many through wishlist_products.	Join columns wishlist_id and product_id.

5. Repository Queries and Database Access

Repository	Method / Query	Purpose
UserRepo	findById, findByEmail, existsByEmail	Used by auth, profile, seller, cart, orders, OTP duplicate checks.
UserRepo	deleteExpiredSellers(status, date)	Custom delete query intended to remove old pending sellers.
ProductRepo	search(key, category)	JPQL search across lower(name), lower(description), lower(brand), with optional
ProductRepo	findByCategory	Supports category pages.
ProductRepo	findBySeller_UserId	Supports seller dashboard product list.
OrderRepo	findByUser_UserId	Loads customer order history.
CartRepo	findByUser_UserId	Loads or creates user cart.
CommentRepo	findByProductId, findByUserUserId, findByUser_UserIdAndProductId	Used by product details, user reviews, and user's review on one product.
CommentRepo	AVG(rating), COUNT(comment)	Calculates product rating summary.
WishlistRepo	findByUser_UserId, existsByUser_UserIdAndProductId	Loads user's wishlist and checks whether a product is saved.
OtpRepo	findByEmail, deleteByEmail	Stores one active OTP per email.
PRRepo	findByToken, deleteByEmail	Stores one reset token per email.
sellerDetailsRepo	findByUser_UserId, findByVerificationStatus	Loads seller profile and admin pending seller queue.

6. Data Lifecycle by Feature

User account: Registration writes users with encoded passwords. Login reads by email. Profile update changes user name, email, phone, address, and pin code.

Seller verification: SellerDetails is created or reused for a seller. New details default to PENDING. Admin approval updates verification_status to APPROVED.

Products: Seller creates Product rows. Stock decides status. Public listing/search reads ProductResponse DTOs. Product images are stored as image_url.

Cart: Cart is created lazily when adding an item. CartItem rows are added, incremented, removed individually, or removed through cart orphan removal.

Orders: Order placement reads products, validates stock, decreases product stock, creates orders and order_item rows, and stores total prices.

Wishlist: Wishlist is created lazily and links products through wishlist_products. Removing deletes only the join table link, not the product.

Comments and ratings: Comment rows store product reviews. Rating endpoints aggregate comment.rating with AVG and COUNT queries.

OTP: Request deletes older OTP for the email, stores a new code and expiry, then sends email. Verify deletes the OTP after success.

Password reset: Request deletes older reset rows, stores a new UUID token, emails link. Reset updates user password and deletes token.

7. Cascades and Deletes

- Deleting a User cascades to orders, comments, and cart because User has cascade=ALL with orphanRemoval on those relationships.
- Deleting a Cart cascades to cart items because Cart.items has cascade=ALL and orphanRemoval=true.
- Deleting an Order cascades to order items because Order.orderItems has cascade=ALL and orphanRemoval=true.
- Deleting a Product cascades to comments because Product.comments has cascade=ALL and orphanRemoval=true.
- Wishlist uses a many-to-many join table; removing from wishlist deletes only the join row, not the product.
- SellerDetails is not mapped back from User in the User entity, but SellerDetails owns a one-to-one FK to users.

8. Database Configuration

- Datasource driver is org.postgresql.Driver.
- The project is configured to connect to a Neon PostgreSQL database with SSL required.
- spring.jpa.hibernate.ddl-auto=update lets Hibernate alter schema from entities during startup.
- spring.jpa.show-sql=true prints SQL statements, useful for development.
- spring.datasource.hikari.auto-commit=false is set, while transactional service methods explicitly control important write flows.
- Credentials and secrets are present in application.properties. The PDF intentionally does not repeat secret values.

9. DTOs Related to Database Data

DTO	Database-facing role
UserRequest/UserResponse	Used for register, profile update, admin/user profile reads.

DTO	Database-facing role
ProductRequest/ProductResponse	Used for product create/update/list/read. ProductResponse excludes image URL and release date.
SellerDetailReq/SellerDetailRes	Used for seller verification submission and status display.
CartResponse/CartItemResponse	Summarizes cart owner, items, prices, quantities, and totals.
OrderRequest/OrderResponse/OrderItemResponse	Represents checkout request and stored order response.
CommentRequest/CommentResponse	Represents review text, rating, optional image URL, reviewer identity, and date.
OtpReq	Carries email and OTP for signup verification.
ResetReq	Carries email, token, and new password for reset flow.
RatingResponse	Carries average product rating and total review count.

10. Important Database Risks and Fix Suggestions

- Hard-coded database and mail credentials exist in application.properties. Move them to environment variables and rotate exposed secrets.
- UserRepo.deleteExpiredSellers compares User.createdOn, a LocalDate, with a LocalDateTime parameter. This should be checked because it may fail or behave unexpectedly.
- Seller product update/delete receives a product ID but ProductService does not verify product ownership. Add a seller ownership check before updates/deletes.
- Comment create/update uses hasRole('USER'), but the UserRole enum has CUSTOMER, SELLER, ADMIN. That likely blocks valid customer review writes.
- Product status can become stale after checkout. OrderService and CartService reduce stock but do not update Product.status when stock reaches zero.
- CommentRepo.getRating returns Double but RatingResponse expects primitive double. If no comments exist, AVG can be null and may cause a null unboxing problem.
- PasswordReset has no expiry column. Reset tokens remain valid until used or replaced by a newer token for the same email.
- OTP has a verified column, but current flow deletes successful OTP records instead of setting verified. The column may be unnecessary.
- No explicit unique constraint prevents duplicate comments from the same user on the same product. The frontend expects one review per user/product, so this may need a DB-level unique constraint.
- Many string fields have no length, validation, or not-null constraints. Add constraints where business rules require them.

11. Suggested Future Schema Improvements

Improvement	Why it helps
Add reset_token_expiry	Add expiry_time to password_reset and reject expired links.
Add product ownership constraints	Check seller ID in update/delete queries or service layer.
Add review uniqueness	Unique index on (user_id, product_id) for comment if one review per customer is desired.
Add audit columns	created_at and updated_at for products, users, orders, comments, seller details.
Normalize categories	Move category from String into a Category table or enum if categories are fixed.
Store order price snapshot	OrderItem.totalPrice exists; consider storing unitPrice too for clearer history.
Improve stock/status sync	Update product status whenever stock changes.
Clean image model	Use consistent image_url return behavior: redirect or JSON URL, not both depending on endpoint.

Generated from local source scan of C:/E-Com/backend/eCom model, repository, service, controller, and configuration files.